La caja negra

Diseño y arquitectura del reemplazo

Documento de diseño técnico para el reemplazo fiel de un servicio HTTP en producción sin código fuente ni documentación.

Principios de diseño

1. Fidelidad del cable sobre elegancia. Las rarezas del original se conservan; no se corrige el protocolo.
2. Pocas piezas. Cada componente adicional es un punto de falla y se agrega solo lo que el sistema necesita.
3. Lo no sondado se nombra como supuesto. No se afirma un comportamiento sin una prueba repetida.
4. El proceso no es la fuente de verdad. El estado sobrevive al reinicio y no depende de qué instancia atiende el request.

1. Contexto y objetivo

Un servicio HTTP lleva años en producción sin código fuente ni documentación. Lo usan clientes reales, mueve dinero real y sostiene operaciones reales. El objetivo es un reemplazo que, ante el mismo request, devuelva el mismo status y el mismo cuerpo que el original, errores incluidos, sin llamarlo.

Además de idéntico en el cable, el reemplazo tiene que ser operable: durable, seguro ante concurrencia, con credenciales revocables y con rastro auditable. Idéntico no alcanza si un reinicio pierde plata: los saldos son de los clientes, no nuestros. Cada token es un cliente con su propio estado, que ningún otro cliente puede leer ni modificar.

2. Restricciones y supuestos

Restricciones del contrato que condicionan el diseño:

- Todas las llamadas llevan `Authorization: Bearer <token>`; con otro token o sin token, el reemplazo responde como el original.
- POST `/msg` recibe `{"m": "COMANDO;args;control"}`; POST `/__reset` vuelve al estado inicial; GET `/health` responde 200 cuando está listo; cualquier otra ruta responde exactamente como el original.
- Un test pasa con el mismo status y el mismo cuerpo: si ambos son JSON, el orden de las claves no importa; si alguno no es JSON, se compara el texto carácter por carácter. Cada paso tiene 5 segundos.
- El reemplazo no puede llamar al original ni a otro reemplazo. Cada token personal admite 10 requests por segundo y 8000 por día contra el original.

Supuestos de diseño

Supuesto	Por qué se asume	Si resulta falso
Hay estado por cliente.	El reset existe y los ejemplos muestran saldos que cambian.	Sin estado el libro sobraría; hoy es la pieza que sostiene saldos y contadores.
El id del cliente sale del token.	Ningún ejemplo trae un id de cliente en el cuerpo.	Habría que leerlo del mensaje y el aislamiento por token no bastaría.

Supuesto	Por qué se asume	Si resulta falso
La cuenta solo existe dentro de un cliente.	El reset es por token y los ejemplos nombran AC- sin dueño.	Si AC-0001 fuera global, la clave sería solo cuenta_id y habría colisiones.
El original distingue una falla interna de un error de negocio.	Producción de años y errores de negocio acotados a un catálogo chico.	Nuestro 503 dejaría de tener sentido y se retiraría; hoy es supuesto.
Una sola región alcanza.	La latencia entre el API y la base entra en el presupuesto de 5 segundos por paso.	Habría que replicar por región y resolver el conflicto de escritura.

3. Arquitectura

Cinco componentes y un proceso aparte. El dominio nunca construye HTTP: devuelve un resultado y el serializador lo convierte en la respuesta exacta del cable. El libro es el único que escribe SQL.

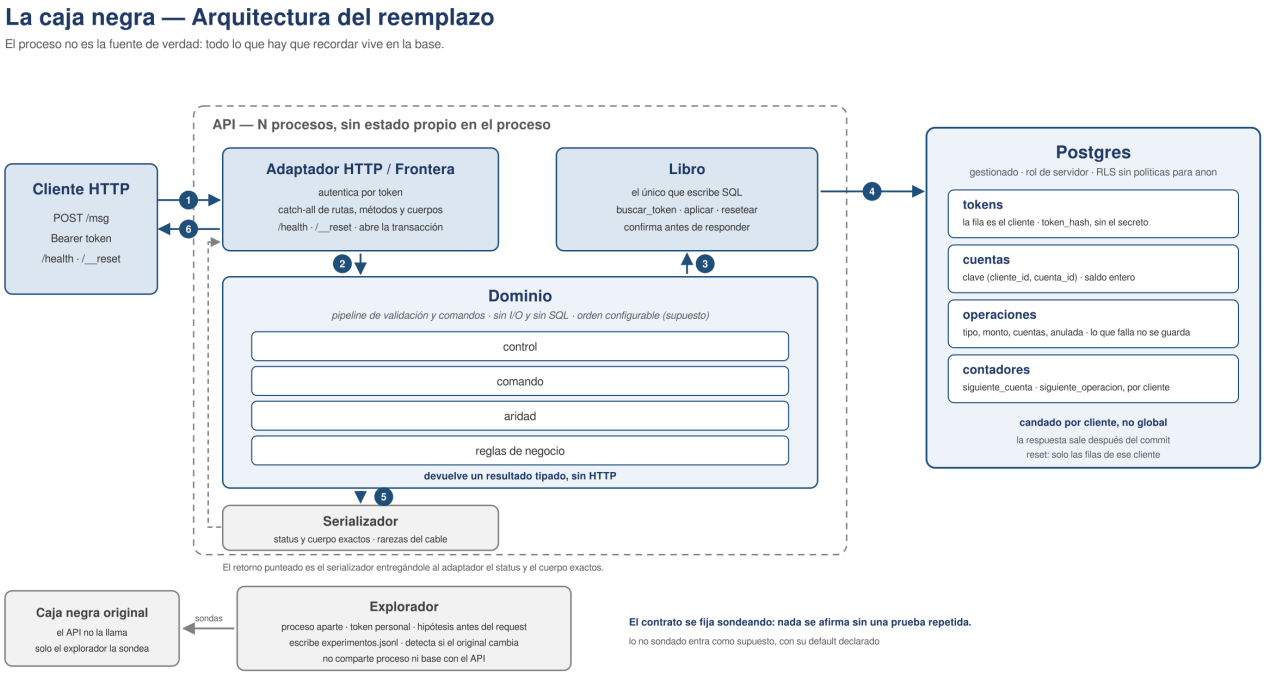


Figura 1 — Arquitectura de componentes

Flujo de un request

1. El cliente envía POST `/msg` con su token; la frontera clasifica la ruta, autentica el token y abre una transacción.
2. El dominio ejecuta el pipeline y el comando contra el cliente autenticado, leyendo y escribiendo por el libro.
3. El libro confirma la transacción y el serializador convierte el resultado tipado en status y cuerpo exactos.
4. La frontera responde al cliente, ya confirmado el commit.

4. Diseño de los componentes

4.1 Frontera / Adaptador HTTP

• Recibe toda ruta y método, autentica por token y no decide reglas de negocio; ningún default del framework llega al cliente.

- Resuelve lo no previsto con tabla propia: método no permitido, tipo de contenido, JSON malformado, falta del campo `m`, barra final, HEAD, OPTIONS y GET sobre /msg.
- Expone `/health` y `/__reset`, abre una transacción por request, delega en el dominio y responde después del commit.

4.2 Dominio

Recibe el mensaje y el cliente autenticado y devuelve un resultado tipado. No conoce SQL y no hace I/O, para poder probarlo sin base.

> **■ Función de control — Riesgo principal**
> Es determinista y función pura del mensaje. No es un nonce ni un contador.
> No distingue mayúsculas. No es un checksum barato: se probaron ~30 funciones offline sobre los 20 ejemplos con cero coincidencias.
> **Default declarado:** aceptar cualquier control numérico mientras no se descubra la fórmula. Es un supuesto explícito.
> El control es la **compuerta**: sin control aceptado no se reproduce ninguno de los 20 ejemplos. Es el paso 1 del orden de implementación.

4.3 Libro y serializador

El libro es el único que escribe SQL. El serializador escribe respuestas literales, nunca construidas por concatenación, y conserva las rarezas del cable (`recieved`, `cuentaId` frente a `cuenta_id`, espacio final del mensaje de comando desconocido). No se corrigen.

Tabla de traducción resultado → respuesta

Resultado	Status	Cuerpo
Pong	200	{"pong": true}
CuentaCreada(id)	201	{"cuentaId": id}
Saldo(id, monto)	200	{"saldo": monto, "cuenta_id": id}
OperacionAceptada(id)	200	{"status": "recieved", "operacionId": id}
ControlInvalido	400	{"error": "E202", "mensaje": "control inválido"}
ComandoDesconocido	400	{"error": "E301", "mensaje": "comando desconocido "}
LimiteExcedido	422	{"error": "E506", "mensaje": "límite excedido"}
SaldoInsuficiente	422	{"error": "E507", "mensaje": "saldo insuficiente"}

4.4 Modelo de datos (cuatro tablas)

Tabla	Clave	Notas clave
tokens	id	token_hash único (HMAC-SHA256 + pepper). No se guarda el secreto.
cuentas	(cliente_id, cuenta_id)	saldo entero, nombre. La clave incluye el cliente.
operaciones	(cliente_id, operacion_id)	tipo, monto, cuentas, bandera anulada. Lo que falla no se guarda.
contadores	cliente_id	siguiente_cuenta y siguiente_operacion. Una fila por cliente.

4.5 Aislamiento, concurrencia y durabilidad

Un token es un cliente. El cliente no manda su id: sale del token vigente. Lo que es de otro cliente no se lee ni se modifica.

- El dominio corre dentro de una transacción por request; la respuesta HTTP sale **después del commit**.
- Candado por cliente (pg_advisory_xact_lock). Dos clientes distintos operan en paralelo; dos movimientos del mismo cliente no se pisan.
- El token se relee dentro de la transacción: si lo revocaron, no se confirma.
- La espera del candado corta a los 4 segundos (el cliente abandona a los 5).
- No se reintenta un POST /msg: un reintento puede depositar dos veces.

5. Decisiones y alternativas descartadas

Decisión	Se descarta	Por qué
Estado en Postgres, no en el proceso.	El mapa en memoria.	Los saldos son plata de clientes reales; un reinicio no puede perderlos.
Candado por cliente.	El lock global.	Dos movimientos del mismo cliente no pueden leer el mismo saldo.
Responder después del commit.	Responder antes de confirmar.	Un 200 sobre una transacción que después falla es plata que el cliente cree que movió.
Registro de tokens en la base con secreto hasheado.	Lista de tokens en variable de entorno.	Se necesita revocar y rotar sin desplegar y sin guardar el secreto.
CLI de administración.	Ruta pública de administración.	El contrato no admite rutas nuevas.
El dominio devuelve un resultado y el serializador escribe el JSON.	Que el dominio arme el HTTP.	Alguien corregiría la rareza del cable.
Saldos leídos de la base, sin caché.	La caché de saldos.	Un saldo viejo es plata mentida.

6. Casos difíciles

Caso	Cómo lo resuelve el diseño
Dos clientes con AC-0001	La clave es (cliente_id, cuenta_id). Para el cliente B, la cuenta de A no existe.
Reset de un cliente	Borra solo las filas de ese cliente_id y devuelve sus contadores a 1.
Dos débitos del mismo cliente	Candado por cliente: el segundo débito ve el saldo ya descontado.
Reinicio del proceso	El estado está en la base; el saldo y los contadores siguen ahí.
El token se revoca a mitad de un request	El token se relee dentro de la transacción; si ya no está vigente, no se confirma.

7. Riesgos y mitigaciones

Riesgo	Mitigación
El proceso se reinicia y pierde el estado.	Cerrado por diseño: el estado vive en la base.
La fórmula del control no se descubre.	Default declarado + sonda como paso 1. El resto bloqueado hasta tenerla.
El orden de errores y límites siguen siendo supuestos.	Mantenerlos como configuración y marcarlos hasta sondearlos.
El candado espera más de la cuenta.	Corte a los 4 segundos y 503.
El protocolo permite un depósito duplicado por reintento.	Riesgo abierto del protocolo. No se cierra sin cambiar el cable.

8. Estrategia de verificación

El harness es un script propio: el servicio no llama al original, el script sí. Cada prueba tiene un umbral explícito y el servicio se da por cerrado solo cuando todas lo cumplen.

Prueba	Esperado	Umbral
Ejemplos (20 requests)	Mismo status y cuerpo que lo grabado	20 de 20
Control	Un control aceptado por paso	Compuerta: sin esto el resto no arranca
Diferencial	Mismo status y cuerpo en cada paso	0 diferencias
Aislamiento y reset	B no ve la cuenta de A; reset solo afecta a A	0 lecturas cruzadas
Concurrencia	El segundo débito ve el saldo ya descontado	0 dobles débitos
Persistencia	Saldo y contadores sobreviven al reinicio	0 diferencias
Latencia	p95 por paso	< 1 s (máx < 5 s)

9. Fuera de alcance

- Llamar al original desde el servicio.
- Corregir `recieved` o unificar `cuentaId` con `cuenta_id`.
- Una ruta pública para administrar tokens.
- Mejorar el protocolo. La igualdad incluye el error y la rareza.

> **Riesgo abierto del protocolo**
> El protocolo no trae una clave de idempotencia del cliente (el OP- lo emite el servicio). Un reintento del cliente puede depositar dos veces. El reemplazo no puede inventar una clave sin cambiar el cable.

10. Orden de implementación

Ordenado por riesgo, no por capas. Primero lo que puede tumbar todo el trabajo.

1. **Función de control** (aislada, con sondas). Es la compuerta.
2. **Esqueleto desplegable** con la base: autenticación, /health, /__reset y los 20 ejemplos.
3. **Catch-all del adaptador** y serializador con respuestas literales.
4. **Dominio:** pipeline, comandos, transacción y candado por cliente.
5. **Registro de tokens y CLI.**
6. **Harness diferencial** y rejuego periódico de los ejemplos.
7. **Ajustar** lo que las sondas muevan: fórmula del control, orden de errores, límites.

Supuestos abiertos (no son reglas)

- Cuerpo de rutas desconocidas, métodos no permitidos, JSON malformado, falta de campo m, etc.
- Cuerpo de /health y de /__reset (con y sin token).
- Orden entre aridad, control y comando; fórmula exacta del control.
- Si el tope es por operación o acumulado.
- 503 cuando fallamos nosotros; que ANULA devuelva el saldo; que un error no consuma el id (probable según ejemplos).